

이진희									
진행 단계	진행 과정	진행 결과	진행 효과	피드백					
데이터 분석	데이터를 전체적으로 분석 함	null 값, 수치별 의미, 각데이터에 텍스트별 의미	데이터를 어떻게 설계하느냐에 따라서 전혀 다른 데이터 양상이 될 것으로 추측이 됨. 이번 과제에서는 비교적 모델보다는 데이터를 어떻게 분류하고 활용하느냐에 따라서 달라질 것으로 예상됨	null수치가 지나치게 많은 피쳐들을 체크(브랜드네임이 제일 많았음) NaN,null의 대체값을 어떻게 할지, 신뢰성이 있는 텍스트로 어떻게 진행 할지 고민해봄. (후후 캐글에서 힌트를 얻고, 데이터 처리를 신경쓰고자 함. item_description에서 나오는 설명들에서'No description yet'이 많은 비율을 차지함. 결국치 포함해서 해당 데이터 처리 어떻게 하느냐에 따라서 중요하게 작용이 될 것이라 추측이 됨.					
데이터 전처리	공통으로 전처리 하는 부분에 대한 내용	price가 0인 로우전체 제거 price를 전체 로그처리 의미있는 숫자인 train_id 칼럼 제거 중복행제거	데이터중, 텍스트 가공 없이 피쳐와 로우에 대한 데이터만으로 처리함. 영향이 없는 단순한 index를 제거하고, 오히려 판단에 혼선을 끼치는 가격 0에 대한 행을 전체적으로 제거함.	회의를 통해서 공통적으로 진행할 전처리를 지정, 1. 그 데이터가 의미가 없어야 하며. 2. 이로인해 데이터가 오염이 없어야 함. 없어도 괜찮을 데이터를 정리해서, 그 데이터를 지움.					
텍스트 전처리	공통으로 텍스트를 전처리를 진행하는 부분에 대한 내용	description에 대한 전처리는 공통적으로 진행	데이터의 전처리를 통해서, 확실한 토큰화 토큰화 과정에서 장어가 되는 부분들을 제거 (대소문자, 공백, 특수문자, 불용어 제거)	불용어에서 특히 부정적인 단어들은 포함을 시키기로 했다. 설명에서 부정적인 표현자체가 가격에 영향을 끼칠 것으로 예측이 되기 때문이다. 토큰화까지 진행하고, 다시한번 문자열로 바꿨는데, 이는 TF/IDF를 진행하기 위함이다.					
데이터 전처리(개인)	새로운 칼럼 추가	텍스트들을 통합해서 새로운 피쳐 생성	name / brand_name / category_name 을 합친 'product_text' name / item_description을 합친 'item_description'	해당 피쳐를 합침으로써, 2가지 효과를 줄 수 있다. (1) 텍스트에서 반복하여 나온 부분을 통해서 중요도를 체크할 수 있다. (가령 반복되는 브랜드나, 특징은 분명 판단하기에 좋은 자료가 될 수 있다.) (2) null로 되어있는 자료나, NaN으로 되어있는 자료들을 합칠 수 있다. 즉시라도, brand_name이 null이더라도, 이름에서 나온 brand로 판단이 가능할 수 있게끔 체크했다. 다만, 그것이 브랜드인지, 카테고리인지 정확한 판단이 불가능 할것이기때, 해당 부분을 포함한 텍스트로 가격이 높은 부분이 비싸게 책정이 되는 부분을 체크해서 학습이 될 것을 예상한다.(No description yet) 또한 공란으로 바꾸고, 대신에 이름이 들어갈 수 있게끔 했다. (3) product_text를 통해서 제품의 정보를 확실히 알 수 있게끔. 그렇게 확실한 가격을 판단할 수 있게 하는 것이 중요하고, name& item_description을 통해서 제품 설명을 비어있거나 부실한 부분은 이름을 통해서 파악하고, 다른 다양한 텍스트들로 힌트를 얻을 수 있게함. 이에 대한 단점도 분명 존재한다. (1) 기존에 있는 명확한 부분이 사라진다. 그래서 카테고리 분류가 생각 보다 텍스트처리가 됐을 때, 다른 데이터와 오염이 될 가능성이 있다. (2) 제목과 설명이 상이한 내용일 경우나, 잘못된은 부분이 있을 경우, 오히려파악이 어려워진다.					
사용을 위한 학습데이터 구분	train_test_split	테스팅을 위해서 따로 데이터를 구분한다.	데이터 구분을 짓고 테스트를 따로 하기 위한.	먼저 데이터를 평가하고 판단하는게 제일 급선무라고 판단. 하지만, 나중에 데이터 정확성 판단을 위해서, 나중에 평가 단계 때, 예초에 데이터 전처리만 완성하고 테스트 트레인 데이터를 나눔으로써, 학습만이 아닌 변수가 있는 데이터를 온전히 보관하고 판단할 예정. 지금은 일단 테스트 데이터와 트레인데이터를 모든 처리를 하고 난 뒤 진행함.					
TFIDF 토큰나이저 진행	두 데이터(product_text, item_description) 데이터 처리	TfidfVectorizer(ngram_range=(1, 2), min_df=3, max_features=150000)	각 텍스트 데이터에서 최대 150,000개의 TF-IDF feature를 생성하였다. 정확한 데이터가 나올 수 있게끔 진행. 단어의 문서 내 출현 빈도(TF)와 전체 문서에서의 희소성(IDF)을 함께 반영하여 각 단어의 중요도를 수치화하였다.	두가지의 공통점은 올라간 상품이 '어떤 상품이나'에 대한 설명이다. 단순히 키워드가 다른 부분인데, 디스크립션은 있는 줄글 설명을 통해 얻는 힌트가 포함되어있다. 그리고 이름/카테고리/브랜드가 합쳐진 product_text는 상품을 대표하는 키워드라고 볼 수 있다. 각 상품에서 자주 등장하면서 전체 상품에서 지나치게 흔하지 않은 단어에 상대적으로 높은 가중치를 부여하여 가격 예측에 활용하고자 하였다.. 학습 데이터에는 fit_transform()를 적용하여 TF-IDF의 vocabulary와 IDF를 학습하고, 검증 데이터에는 학습된 Vectorizer의 transform()만 적용하였다. 이를 통해 검증 데이터의 정보가 학습 과정에 포함되는 데이터 누수를 방지하였다.					
희소행렬을 어떻게 처리할 것인가.	tfidf확인 후, 해당 관련 transform 진행하고, 나머지 item_condition_id, shipping도 희소행렬로 같이 넣기	csr_matrix이후, hstack으로 하나로 합침.	거대한 TF-IDF 희소행렬에 숫자 피쳐 2개를 붙이면서 최종 데이터도 희소행렬 형태로 유지하는 것	효율적 → 30만 차원의 TF-IDF를 dense로 풀지 않고 sparse 상태로 계속 처리하므로 메모리를 절약할 수 있음. 명확함 → hstack에 들어가는 세 데이터가 모두 sparse matrix이므로, "희소행렬 3개를 합친다"는 구조가 코드상 분명함. TF-IDF가 희소행렬이므로 수치형 피쳐도 csr_matrix로 변환하여 동일한 희소행렬 형식으로 맞추은 뒤 hstack으로 결합하였다.					
릿지 돌리기	ridge = Ridge(alpha=1.0)	이후, rmse로 체크(Target인 price에 log1p 변환을 선행했기 때문에, 로그 변환된 실제값과 예측값의 RMSE를 계산하였다. 이는 RMSLE와 동일한 형태의 평가가 된다.)	Validation RMSLE: 0.4375787842324169	릿지로 해서 일단 최대한 빠르고 간단하게 돌려봄. 생각보다 효과적으로 나왔음					

릿지 파라미터 정리	for alpha in [0.1, 0.5, 1.0, 2.0, 5.0]: ridge = Ridge(alpha=alpha) ridge.fit(X_train_final, y_train) pred = ridge.predict(X_valid_final) rmsle = np.sqrt((mean_squared_error(y_valid, pred)))	0.1 0.4589190979559455 0.5 0.4426501853498952 1.0 0.4375787842324169 2.0 0.43546833927182665 5.0 0.4382149650974841	alpha=2.0에서 RMSLE가 0.43547로 가장 낮게 나타나 가장 좋은 성능을 보였다.	물론 3.4도 비교해봐야 정확할 수 있지만, 기본적으로 알파수치를 비교해서 가장 값이 잘나온 수치를 비교했다. 더 좋은 값을 나오기 위한 파라미터 조정을 했다. alpha가 0.1에서 2.0까지 증가하면서 성능이 개선되었으나, 5.0에서는 다시 RMSLE가 증가하였다. 이를 통해 지나치게 약하거나 강한 규제보다 적절한 수준의 L2 규제가 일반화 성능에 유리함을 확인하였다.						
데이터 함수화	반복적으로 사용할 수 있는 것들은 함수화	data전처리에 대한 내용을 함수로 진행	함수화를 통해서 추후, 데이터스플릿을 먼저 진행하게 될때, 함수를 통해서 진행할 예정	반복하게 되는 기능들을 정리하고 반복사용할 수 있도록 함수로 만들었다.						
데이터 전처리(후편)	rm- 마스킹 처리가 된 description 불용어처리. price 2000이상 row 제거	데이터 학습에 문제생길 부분에 대해서 점검하고 추가로 제거할 내용을 제거.	2000이상의 가격열 제거 rm이 반복되는데, 마스킹으로 되어있는 부분 상품과 관련없으므로 삭제	용우님의 아이디어로 적용하게 된 부분, Mercari에서 허용하는 상품 가격 상한이 \$2,000이므로, 이를 초과하는 값은 정상적인 상품 가격 범위를 벗어난 데이터라고 판단하여 제거하였다.(또한 가격 분포를 확인한 결과 2,000 이상 값이 극단적인 고가 영역에 위치하여 모델 학습에 과도한 영향을 줄 가능성이 있다고 판단해 이상치로 처리하였다.). 그리고 rm이 description 중간에 있는데, 개인정보로 지워진 데이터이기 때문에, 상품과 연관이 없음. 제거						
데이터 전처리(후편)	description 숫자가 제거되어 있었는데, 다시 제거하지 않고 진행하기로	# 영문자와 공백을 제외한 문자 제거 text = re.sub(r"[a-z0-9\s]", '', text) 원래는 a-z만 적용.	description 숫자 포함시킴.	상품명과 설명의 숫자는 모델명, 용량, 사이즈, 세대 등 상품 가격을 구분하는 정보가 될 수 있다고 판단하였다. 따라서 기존 정규식에서 제거되던 숫자를 보존하도록 전처리 방식을 수정하였다.						
릿지 똑같은 조건으로 진행	똑같은 조건진행(알파 3.0 추가)	Validation RMSLE: 0.4357978914817806 "0.1 0.4557548155812145 0.5 0.4401367775632817 1.0 0.4357978914817806 2.0 0.43435494059237906 3.0 0.435058833031306 5.0 0.4376482547577345"		Mercari에서 허용하는 상품 가격 상한이 \$2,000이므로, 이를 초과하는 값은 정상적인 상품 가격 범위를 벗어난 데이터라고 판단하여 제거하였다. 가격 분포에서도 해당 데이터가 극단적인 고가 영역에 위치함을 확인하였다.						
TFIDF 토큰나이저 단어보다 글자로	TfidfVectorizer(analyzer='char', ngram_range=(3, 5), min_df=3, max_features=100000, dtype=np.float32)	Validation RMSLE: 0.8980667237837484	글자 TFIDF의 경우, rmsle 수치가 더 안좋게 나온다.	해당 부분에 대한 추측은 여러가지 생각해볼 수 있다. 첫번째로 글자수의 경우, 상당히 많은 피처가 만들어진다. 그 피처들에서 의미를 부여하기에는 많은 공통적인 부분들에 대한 데이터 왜곡이 일어날 수 있다. Character n-gram은 ing과 같이 상품의 의미를 직접적으로 설명하지 않는 문자 조각도 대량으로 생성될 수 있다. TF-IDF가 전체 문서에 흔한 패턴의 가중치를 낮추기는 하지만, 제한된 feature 수 안에서 Word n-gram보다 상품 의미를 직접적으로 표현하는 데 불리했을 가능성이 있다. 물론 max_features를 늘리거나, 상위 단어들을 일부러 지우는 방법도 있지만, 진행하지 않았다. Character n-gram은 Word n-gram보다 생성 가능한 feature 수가 매우 많아 세부적인 문자 패턴을 표현할 수 있으나, 메모리 사용량을 고려하여 max_features=100,000으로 제한하였다. 그 결과 Validation RMSLE가 0.8981로 Word TF-IDF의 0.4355보다 크게 낮은 성능을 보였다. Character TF-IDF의 충분한 표현력을 확보하려면 더 많은 feature가 필요할 가능성이 있으나, 실제 생성된 희소행렬에서도 약 1.74억 개의 non-zero 값이 발생하여 메모리 비용이 크게 증가하므로 추가 확장은 진행하지 않았다.						
LGBM 사용 LGBMRegressor(n_estimators=200, learning_rate=0.1, num_leaves=31, max_depth=6, random_state=42, n_jobs=-1)	Tfidf - max_features=50000 product_text의 길이 단어수, description의 단어수와 길이 수 피처 추가	LightGBM Validation RMSLE: 0.5365198708629355	점수가 그래도 갑자기 좀 높은 수치가 나옴. 캐글에서 처음에 참고했던 Mercari 2위 솔루션의 알고 연결됩니다. 그 팀도 같은 데이터 표현으로 모델만 여러 개 만드는 것이 효과적이지 않았다고 판단하고 서로 다른 입력 표현을 가진 모델들을 사용했다고 했다. 그래서 릿지에서 사용한 데이터셋만으로 세팅해서 똑같은 데이터 셋으로 다시한번 체크해볼 예정.	LGBM의 점수가 생각보다 높아진 것을 확인. 이에 대한 요인은 여러가지로 분석할 수 있는데, (1) 파라미터가 해당 데이터를 담아내지 못할 경우. 파라미터를 설정했을 때, n_estimators 가 200개라서 시행횟수에 부정확성이 있을 수 있음. (2) 그외에 파라미터가 악영향을 끼쳤을 경우 이부분은 일단 상정하지 않기로 하지만, 염두에 두어야 한다고 생각했다. 약 10만개의 피트로 판단하기에는 너무 적은 가짓수일 수 있기에, 이부분을 체크해볼 필요가 있다. (3) 피처 단어수/길이수 추가한 부분. 이부분은 수정할 수 있는 부분 중에서 제일 간단하게 진행할 수 있는 부분이기때 적용해보려고 한다. 똑같이 tfidf데이터를 쓴거로 판단하여 정확한 비교를 해보는 것이 좋다고 생각했다.						
product_text의 길이 단어수, description의 단어수와 길이 수 피처 제거	제거 후, 다시한번 LGBM진행	LightGBM Validation RMSLE: 0.5382020797729394	점수가 다시 높아진 것을 보아, product_text의 길이와 단어수, description의 단어수와 길이가 영향을 끼칠 수 있다.	0.002차이로 크게 차이가 있다고 볼 수는 없다. 하지만, 어느 정도 영향을 끼치는 파라미터라고 볼 수 있다고 생각했다. 그렇기에, 추후 다시 체크해볼 때, 이 데이터를 활용해보고자한다.						

하이퍼 파라미터 조정.	오래걸리더라도, 정확성을 위해서 한번 더 체크	<pre> lgbm = LGBMRegressor(objective='regression', n_estimators=2000, # num_boost_round=2000 # learning_rate=0.1, num_leaves=31, # feature_fraction=0.8, # colsample_bytree=0.8 # bagging_fraction=0.8, # 또는 subsample=0.8 # bagging_freq=5, # 또는 subsample_freq=5 # random_state=42, n_jobs=-1, verbosity=-1) </pre>	111분걸려서 결과도출 Did not meet early stopping. Best iteration is: [2000]training's rmse: 0.422169training's l2: 0.178227valid_1's rmse: 0.441616valid_1's l2: 0.195025	일리스타핑이 되지 않아서, 더 좋은 결과를 도출 할 수 있는 상황. 더 좋은 결과가 나오게 하기 위해서, 좀 더 진행횟수를 늘려서, 방법을 찾아가야한다고 생각했다. 처음에는 0.422가 나와서 좋아했지만, 결국에는 선별과정에서 0.44로 나왔기 때문에, 이를 염두에 두고 과정을 진행해보고자 한다. 일단 LGBM에서 2000개가 아닌 4000개로 진행해면서 혹시라도 최적의 값이 나올지에 대한 체크를 해보겠다.					
하이퍼 파라미터 조정	오래걸리더라도, 정확성을 위해서 한번 더 체크(2) n_estimators=4000	결과가 일리스타핑이 안됐기 때문에 파라미터를 더 늘려서 조정해볼 예정. 그리고 이에 대한 피쳐 임포턴스 적용	200 분 걸려서 결과 도출 Did not meet early stopping. Best iteration is: [4000] training's rmse: 0.396435 training's l2: 0.157161 valid_1's rmse: 0.430869 valid_1's l2: 0.185648	4,000회까지 학습을 진행했으나 Early Stopping은 발생하지 않았으며, 반복 횟수가 증가할수록 Validation RMSLE가 지속적으로 개선되는 모습을 확인하였다. 따라서 6,000회까지 추가 학습을 진행해 볼 여지가 있다고 판단하였다. 다만 다음과 같은 이유로 추가 학습은 후순위로 두었다. (1) LightGBM의 학습 시간이 길어 반복 횟수를 추가로 증가시키는 데 상당한 시간이 필요하다. (2) 학습 횟수가 증가하면서 Training과 Validation의 성능 차이도 점차 커지고 있다. 현재까지는 Validation 성능 역시 계속 개선되고 있어 본격적인 과적합으로 인해 성능이 악화될 상태라고 볼 수는 없지만, Train-Validation 간 격차가 증가하고 있다는 점에서 과적합 영향을 확인할 수 있다. 따라서 현재는 4,000회 학습 결과를 사용하고, 이후 시간적 여유가 있을 경우 6,000회까지 추가 학습하여 Validation 성능의 개선 여부와 Early Stopping 발생 여부를 확인하고자 한다.					
릿지: 브랜드 네임 AND brand_and로 변경	1번 문서에서 수정.(릿지 단어로 처리한 결과)	원래 값 :1.0 0.4357978914817806 Validation RMSLE: 0.4357883640216876 0.1 0.45573734139008526 0.5 0.44012818787874225 1.0 0.4357883640216876 2.0 0.4343426779214986 3.0 0.4350468421404694 5.0 0.4376358037859143	AND브랜드를 and로 불용어처리가 되기전에 brand_and로 바꾸어 불용어가 되는 것을 막았다. 이를 통해서 데이터의 작은 개선을 만들어냈다.	AND라는 브랜드명이 일반 접속사 and와 동일한 토큰으로 처리되는 문제를 확인하여, brand_name에서 발견되는 and를 brand_and로 변경하였다. 총 1,933개의 brand_name에서 and가 발견되었다. name에도 and가 포함될 수 있으나, pink and black과 같이 일반적인 접속사로 사용되는 경우와 브랜드명을 구분하기 어렵기 때문에 brand_name에 한정하여 변환하였다. 변경 후 Ridge 모델을 다시 학습한 결과, 최적 alpha=2.0에서 Validation RMSLE가 기존 0.4343549에서 0.4343427로 약 0.0000123 감소하였다. 성능 개선 폭은 매우 작지만, 동일한 단어라도 데이터의 의미에 따라 토큰을 구분하는 전처리가 실제 예측 결과에 영향을 줄 수 있음을 확인하였다.					
릿지: 맥스 피쳐스 변경	product_tfidf = TfidfVectorizer(ngram_range=(1, 2), min_df=3, max_features=200000)	원래 값에 위에 값을 참조 Validation RMSLE: 0.4340316806663665 0.1 0.4607866178125189 0.5 0.4392468750565992 1.0 0.4340316806663665 2.0 0.4324559593054598 3.0 0.43331148294723765 5.0 0.43624025026369806	max_feature를 20만개로 진행함으로써, 모델이 학습할 수 있는 단어 개수를 늘려보았다. 상당히 유의미한 결과를 불러왔다.	약 0.4퍼센트의 개선이 이뤄졌다. 다른 결과에 비해서 확연하게 이뤄지는 변동이라고 볼 수 있었다. 여전히 숨겨진 단어들이 결국 가격에 더 많은 영향력을 끼칠 수 있다는 부분을 점검할 수 있고, 그안에서 필요한 단어들을 학습할 수 있다는 장점이 있다. 그렇다면 오히려 max_feature가 낮아졌을 때는 점수가 안좋아질 것인가에 대한 의문이 들었다. 다시한번 체크해보고 진행하려고 한다.					

릿지: 맥스 피쳐스 변경	product_tfidf = TfidfVectorizer(ngram_range=(1, 2), min_df=3, max_features=100000)	Validation RMSLE: 0.4383581223084487 0.1 0.44999329583456243 0.5 0.4412536380975099 1.0 0.4383581223084487 2.0 0.43734144514072437 3.0 0.43793658815891534 5.0 0.4401350631463877	max_features를 100,000, 150,000, 200,000으로 변경하여 성능을 비교하였다. 동일한 alpha=1.0 기준에서 Validation RMSLE는 각각 0.438358, 0.435788, 0.434032로 나타났으며, 피쳐 수가 증가할수록 성능이 개선되는 모습을 확인하였다. 다만 100,000개에서 150,000개로 증가했을 때의 RMSLE 개선폭은 약 0.00257이었던 반면, 150,000개에서 200,000개로 증가했을 때는 약 0.00176으로 개선폭이 감소하였다. 이를 통해 현재 범위에서는 피쳐 수 증가가 성능 개선에 기여하지만, 피쳐 수가 계속 증가할수록 추가적인 성능 향상 폭은 점차 제한될 가능성이 있다고 판단하였다. 또한 피쳐 수 증가는 메모리 사용량과 학습 비용을 증가시키며, 불필요하거나 희귀한 피쳐가 추가될 경우 과적합 가능성도 고려해야 한다. 현재 Validation 성능에서는 과적합으로 인한 악화가 관찰되지는 않았으나, 계산 비용과 성능 개선폭을 함께 고려하여 max_features에 대한 추가 증가는 진행하지 않고 200,000개까지의 실험으로 마무리하였다.	150000개에서 200000개로 했을 때의 등락폭은 0.0017, 150000개 기준으로 100000개로 낮췄을 때의 등락폭은 0.0026이었다. 물론 해당 수치로만 무조건 판단할 수 없지만, 점차 개수가 많아진다고 해서 무조건 효과가 있어진다는 점은 아니라는 점을 알 수 있다. 점차 개수를 늘려도 크게 차이를 못느끼는 단계까지 올 수 있다. 그렇기에 이번엔 테스트는 여기까지 진행할 예정이다. 첫번째로 아무리 릿지로 돌린다고 하더라도, 결국엔 피쳐가 많아질 수록 성능은 과적합의 문제도 있기 때문에, 무조건 이것이 정답이라고 이야기할 수 없는 단계로 갈 수 있고, 두번째로 아까 이야기했듯이 해당 추세가 그대로 이어진다면 25만개의 데이터로 학습을 한다고 하더라도, 큰 수치변화를 느낄 수 없을 것이다. 데이터가 많아지면, 메모리에 문제가 생긴다는 점까지 감안했을 때, max_feature 조정은 여기까지 진행해볼 예정이다.				
맥스 피쳐스 재조정	product_text와 item_description의 데이터 변경 최적의 수치를 체크하기	150000/200000-Validation RMSLE: 0.434929294008602 100000/200000-Validation RMSLE: 0.4361620220074503 200000/150000-Validation RMSLE: 0.43480927014773707	따로 200000/100000은 하지않았다. 200000/200000이 현재 나올 수 있는 최선의 값임을인지했다.	생각보다 조금씩 차이가 있지만, 검색한 값의 제일 최적의 값은 200000/200000으로 고정시킬 예정이다. 200000/100000은 추세상 오히려 안좋게 나올 가능성이 있기 때문에, 검사를 진행하지 않기로 했다. 물론 그외에 200000이상의 값, 혹은 그 사이값을 넣어서 더 검색할 수 있지만, 어디까지나 경향성을 보고, 앞선 설정대로 계산 비용과 성능 개선폭을 염두에 두어 mindf를 검사해보기로 했다.				
릿지: min_df 변경	product_tfidf = TfidfVectorizer(ngram_range=(1, 2), min_df=5, max_features=200000) (현재 max_features를 20만으로 체크하고진행) 릿지 알파값은 1.0 기준	기준값: 0.4340316806663665(3/3) 5/5 =Validation RMSLE: 0.43509623052436774 3/5 =Validation RMSLE: 0.43407374662640863 5/3 =Validation RMSLE: 0.43509883875169886 2/3 = 0.434062806499	오히려 더 낮은 수치가 나왔다. 수치가 낮을 수록 더 좋은 판단기준이 될 수 있다는 것을 알 수 있다.	min_df 값을 조정하여 낮은 빈도로 등장하는 단어를 제거했을 때 모델 성능이 어떻게 변화하는지 확인하였다. 기준값인 product_text=3, item_description=3에서는 Validation RMSLE가 0.434032로 가장 낮게 나타났다. 두 값을 모두 5로 증가시킨 경우에는 0.435096으로 성능이 하락하였다. 이를 통해 단순히 등장 빈도가 낮은 단어를 더 많이 제거한다고 해서 모델의 성능이 향상되는 것은 아니라는 점을 확인하였다. 각 텍스트를 개별적으로 확인했을 때도 차이가 나타났다. product_text의 min_df만 5로 증가시킨 5/3에서는 RMSLE가 0.435099로 증가하여 성능이 비교적 뚜렷하게 하락하였다. product_text에는 상품명, 브랜드명, 카테고리과 같은 정보가 포함되어 있기 때문에 등장 빈도가 낮은 단어 중에서도 특정 브랜드나 상품을 구분하는 데 유용한 정보가 포함되어 있을 가능성이 있다고 판단하였다. 반면 item_description의 min_df만 5로 증가시킨 3/5에서는 RMSLE가 0.434074로, 기존 3/3의 0.434032와 비교하여 약 0.000042의 작은 차이만 나타났다. 따라서 description에서 빈도가 낮은 일부 단어를 제거하는 것은 product_text에 비해 모델 성능에 미치는 영향이 상대적으로 작았다고 볼 수 있다. 현재 결과에서는 min_df=3이 가장 좋은 성능을 보였으며, 특히 product_text의 희귀 단어를 지나치게 제거하지 않는 것이 중요할 가능성을 확인하였다. 그렇다고 해서 product_text를 min_df로 한다고 해서 더 좋은 결과가 나온 것은 아니다. 다소 작은 차이로 정수가 더 안좋게 나왔기 때문이다.				
Ridge + 길이/단어 수 Scaling	프로덕트 텍스트 / 디스크립션의 길이 / 단어 수 피쳐 생성 및 스케일링	Validation RMSLE: 0.43313235538671757	프로덕트 텍스트와 디스크립션의 길이 / 단어 수 자체가 데이터에서 가격 예측에 유용한 정보를 제공했다	그 결과 Validation RMSLE가 기존 0.434032에서 0.433132로 약 0.000899 감소하였다. 이를 통해 TF-IDF로 표현한 단어의 종류와 중요도 뿐만 아니라, 상품명 및 설명문의 길이와 단어 수와 같은 텍스트의 구조적 정보 역시 가격 예측에 추가적인 정보를 제공할 수 있음을 확인하였다. 다만 이번 결과만으로도 텍스트가 길수록 가격이 높거나 낮다고 해석할 수는 없으며, 길이 및 단어 수 피쳐의 추가가 모델의 예측 성능 향상에 기여했다는 수준으로 해석하였다.				
릿지와 LGBM 앙상블								
데이터 갈무리 test데이터 분리 프라이스 로그 지우고 체크								

추가 LGBM- 6천번의 시도 진행해보기																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																			
------------------------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--